

CMPT310 Project Write-up

Group 4 - RiftPredict

1. System Explanation: (~1 page)

What the system does

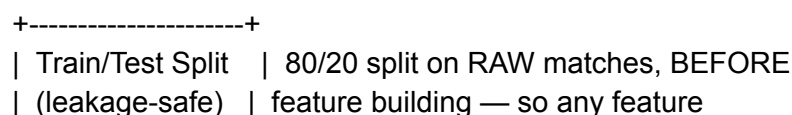
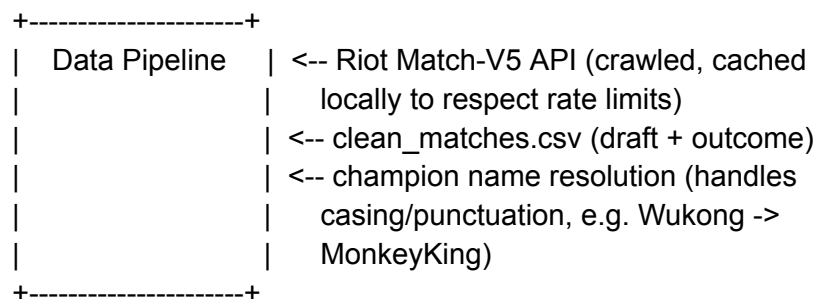
RiftPredict predicts the winner of a League of Legends match from the champion draft alone. A user enters all 10 champion picks (5 per team, in Top/Jungle/Mid/ADC/Support order) through a command-line interface. The system converts that draft into a numeric feature vector, feeds it into a trained classifier, and returns a predicted winner, a win probability for each side, and an explanation of each team's strengths, weaknesses and lane matchups.

AI methods used

RiftPredict frames match prediction as binary supervised classification. We trained and compared three classical classifiers: K-Nearest Neighbors, Decision Tree, and Random Forest. These were combined into a soft-voting ensemble that averages each model's predicted probabilities. The ensemble was selected as the production model because it gives smoother, better-calibrated probability outputs than any single model alone (see Limitations, below).

AI pipeline:

User Input (10 champion picks: Top/Jungle/Mid/ADC/Support x2)



| computed from match history (e.g.
| champion win rate) never sees test data
+-----+



+-----+	
Feature Engineering	<---> champions.json
	- champion tags
- tag counts	+-----+
- champion win rate	+-----+
(Bayesian-smoothed)	<---> feature_matrices.json
- role frequency	- AP ratio / variance
- lane matchup	- role frequencies
advantage	- counter_matrix
+-----+	



+-----+	
Classifiers	KNN Decision Tree Random Forest
	combined via soft-voting Ensemble
+-----+	



+-----+	
Trained Model	best_model.joblib
	+ saved champion win rates
+-----+	



+-----+	
CLI Interface	validates picks - rejects invalid
(user_interface.py)	names, blocks duplicate champions
	across the whole match
+-----+	



Final Prediction
(predicted winner, win probability, team strengths/
weaknesses,)

Limitations:

- **Accuracy is limited for a binary supervised classification problem (52%).** This is common for *draft-only* predictions. Modern League of Legends draft-only model's accuracy is typically around 55%, because individual player skill and in-game decision-making contribute to match outcomes far more than which champions were picked.
- **Small/noisy per-champion statistics.** With ~173 champions across 5 roles and 12000 training matches, some champions have very few appearances in the training split, making naive win-rate estimates unreliable. We mitigated this with Bayesian smoothing, pulling each champion's win rate toward the global average in proportion to how few games they have, rather than trusting a raw win/games ratio computed from a handful of samples.

2. Feature Table: (1–2 pages)

Description	Platform	Completeness	Code	Author(s)	Notes
Feature engineering: (tags, win rate, AP ratio, role freq, lane advantage)	Local	3.5	Python	Ken	Iterated multiple times; removed low-signal static base stats in favor of empirical/matchup features.
Preprocessing: maps champion names to numerical stats, outputting training data	Local	4	Python	Oliver, Eugene	Covers edge cases such as casing, punctuation and nicknames. If a champion has multiple names, unify into one. (ex. Wukong, Monkey King -> Wukong)
Command Line Interface: user inputs all 10 champions -> outputs prediction	Local	5	Python	Oliver, Ken	User inputs champions one at a time. Only moves to the next champion once a valid one is input.
Riot API Data Crawler: Collects ranked matches, handles rate limits, duplicates, and checkpoints.	Local	5	Python	Mark	Riot Games Match-V5 API utilizing a custom matchcrawler that stores only ranked solo/duo matches.
Dataset Cleaning & Validation: Rejects incomplete matches and creates cleaned	Local	5	Python	Mark	Before a match is accepted, the collector verifies that it belongs to the requested queue, contains a valid winner, and has all ten

dataset					required champion-role selections.
Models: KNN / Decision Tree / Random Forest classifiers	Local	5	Python (scikit-learn)	Eugene	Trained and evaluated side by side.
Soft-voting Ensemble	Local	5	Python (scikit-learn)	Ken	Averages KNN/DT/RF probabilities; used as production model for smoother calibration.
Explanation generation: (strengths/weaknesses + lane matchups)	Local	4	Python	Oliver	Generate an appropriate explanation based on the average stats of each team and standard. (ex. average ap>standard: too much ap)
Evaluation & visualization (metrics, confusion matrices, charts)	Local	4	Python (matplotlib, seaborn)	Eugene	Generates metrics.csv per-model confusion matrices, comparison and feature-importance charts after training the models.
Unit Tests	Local	4	Python	All	Test each feature operates as intended.

3. External Tools & Libraries: (~½ page)

- **Datasets & APIs:** Riot Games API, Riot Data Dragon
- **ML Frameworks:** Scikit-learn, Joblib
- **Visualization:** Matplotlib, Seaborn
- **Data Processing:** Pandas, NumPy

Platform dependencies: Python 3.11+, pip install -r requirements.txt (scikit-learn, pandas, numpy, matplotlib, seaborn, joblib, requests). Requires a Riot Games API key (see README) to run the crawler; the trained model and cached match data can be used without one.

Setup:

```
# Clone and enter project
git clone https://github.com/nartnek/RiftPredict.git

cd RiftPredict

# Create virtual environment
python3 -m venv venv          # macOS/Linux
# python -m venv venv         # Windows
```

```
# Activate
source venv/bin/activate          # macOS/Linux
# venv\Scripts\activate          # Windows

# Install dependencies
pip install -r requirements.txt

# Configure API keys
cp .env.example .env             # macOS/Linux
# copy .env.example .env        # Windows
# Edit .env with your API key
```

Running:

```
# Build features and compute champion win rates
python -m src.preprocessing.encode_champions

# Train and evaluate the models
python -m src.models.train_and_evaluate

# CLI to run predictions
python -m src.ui.user_interface

# Run tests
python -m pytest tests/ -v
```

Acknowledgment: All code in this project is original. No open-source code was adapted or copied.